Voice Agent MVP — Design Doc & Event Schema (M1)

Rishu Singhal · Draft v0.1 · Oct 23, 2026 · Status: for self-review, then finalize Oct 24

1. Goal and non-goals

Goal. A user opens a web page, clicks Start Call, speaks naturally, and hears a spoken reply within ~1.5 s of finishing their sentence. Context is kept across turns until End Call.

Non-goals (out of scope per the assignment). Telephony, auth, multi-language, voice cloning, horizontal scaling, analytics dashboard.

Success criteria. The 10 acceptance criteria in the assignment PDF. The ones that drive design: median end-of-speech to first agent audio ≤ 1.5 s, barge-in stops audio within ~300 ms, providers swappable by config + adapter only.

2. Key decisions (confirm each one before coding)

#	Decision	Proposed choice	Why
D1	Backend language / framework	Python 3.11 + FastAPI + asyncio	Native WebSockets, best audio/AI library support, easy cancellation with asyncio tasks
D2	Client → server audio	Raw PCM, signed 16-bit little-endian, 16 kHz, mono, 20 ms frames (640 bytes)	Matches VAD and STT input directly; no codec on the hot path
D3	Server → client audio	PCM s16le, mono, sample rate chosen by the TTS adapter and announced in `agent_audio_start`	Avoids resampling inside the pipeline; the client plays at the announced rate
D4	VAD	Silero VAD (or WebRTC VAD) running server-side on 20–30 ms frames	Open source, local, no quota; end-of-utterance silence is configurable (default 600 ms)

#	Decision	Proposed choice	Why
D5	STT	Streaming provider behind `STTAdapter` (e.g. Deepgram free credits); local fallback (Vosk / faster-whisper)	Partials needed for captions
D6	LLM	Streaming provider behind `LLMAdapter` (e.g. Gemini or Groq free tier)	Low first-token latency, tool calling
D7	TTS	Streaming or per-sentence provider behind `TTSAdapter`; local fallback (Piper)	Sentence-level synthesis is enough for the latency budget
D8	Persistence	SQLite	Zero setup, enough for per-session transcript and metrics
D9	Barge-in detection	Client-side energy threshold while the agent is speaking, with browser echo cancellation on	Fastest path to the ~300 ms target; server stays the source of truth and cancels work

The providers in D5–D7 are placeholders until free-tier access is checked. The adapter design means the choice does not change the rest of this doc.

3. Architecture

Browser (mic capture via AudioWorklet, playback queue, captions, status, Start/End) talks to the backend over one WebSocket per call. Backend modules, each behind an interface:

- **Session Manager**: creates and ends sessions, holds history and state, owns timeouts.
- **VAD**: audio frames in, speech start / speech end events out.
- **STT adapter**: audio in, partial and final transcripts out.
- **Agent (LLM adapter + prompt builder)**: transcript + history in, token stream and tool calls out.
- **Tool Executor**: registry of tools; runs a tool call and returns a JSON result or an error.
- **TTS adapter**: text in, audio chunks out.
- **Sentence Buffer**: groups streamed tokens into sentences for TTS.

- **Metrics / Persistence**: per-turn stage timestamps, transcript, errors written to SQLite.

Adapter interfaces (pseudo-Python)

```
class VADProcessor(Protocol):
    def process(self, frame: bytes) -> VADEvent | None: ...   # SPEECH_START
| SPEECH_END

class STTStream(Protocol):
    async def send_audio(self, pcm: bytes) -> None: ...
    def events(self) -> AsyncIterator[STTEvent]: ...          # Partial(text)
| Final(text)
    async def finish(self) -> None: ...

class STTAdapter(Protocol):
    async def open_stream(self, sample_rate: int) -> STTStream: ...

class LLMAdapter(Protocol):
    def stream(self, messages: list[Message], tools: list[ToolSpec]) ->
AsyncIterator[LLMEvent]: ...
    # TextDelta(text) | ToolCall(id, name, args) | Done(usage)

class TTSAdapter(Protocol):
    sample_rate: int
    def synthesize(self, text: str) -> AsyncIterator[bytes]: ...   # PCM
chunks

class Tool(Protocol):
    spec: ToolSpec
    async def run(self, args: dict) -> dict: ...
```

Rules: the pipeline only imports these protocols. Concrete adapters live in `adapters/` and are selected from config by a small registry (`STT_PROVIDER=deepgram`). Swapping a provider means a new adapter file plus a config change. Cancellation is done by cancelling the asyncio task that is iterating the adapter stream, so every adapter must be cancel-safe.

4. Session state machine

States: `IDLE`, `LISTENING`, `THINKING`, `SPEAKING`, `ENDED`.

From	Event	To	Side effects
IDLE	`POST /sessions` + WS connected	LISTENING	Start silence timer, open STT stream
LISTENING	VAD speech_start	LISTENING	Forward frames to STT, reset silence timer
LISTENING	VAD speech_end with valid utterance	THINKING	Finalize STT, stamp `vad_end_ms`
LISTENING	VAD speech_end with noise-only or too-short utterance	LISTENING	Drop; no LLM call
THINKING	`transcript_final` received	THINKING	Append to history, start LLM stream
THINKING	First sentence ready for TTS	SPEAKING	Start streaming audio to client
THINKING	Tool call requested	THINKING	Run tool, feed result back to LLM
THINKING	STT / LLM / TTS error or timeout	SPEAKING	Speak fallback line, log error
SPEAKING	Reply finished and playback done	LISTENING	Commit full agent turn to history
SPEAKING	`interrupt` from client	LISTENING	Cancel LLM and TTS tasks, keep only the spoken part in history
any	End Call / 2 min silence / max duration / WS drop	ENDED	Say goodbye if socket alive, send `call_ended`, persist, close

Timeouts (all configurable): silence 120 s, max call duration (default 10 min), STT final 3 s, LLM first token 5 s, TTS first audio 5 s.

5. Turn lifecycle (one turn)

1. Client streams 20 ms PCM frames. Server VAD marks speech start.
2. Frames go to the STT stream; partials are sent as `transcript_partial`.

3. VAD marks speech end after the silence threshold. Server stamps `vad_end_ms`, asks STT for the final transcript, sends `transcript_final`, and sets state to THINKING.
4. Agent builds the prompt (system prompt + history + transcript) and streams tokens. A tool call pauses the stream, runs the tool, and resumes with the result.
5. Sentence Buffer cuts the stream at sentence boundaries. Each sentence is sent as `agent_text` and queued to TTS immediately.
6. TTS audio chunks go out as binary frames between `agent_audio_start` and `agent_audio_end`.
7. When the last chunk has played, the client sends `playback_done` and the server returns to LISTENING.

Barge-in

- While `state = speaking`, the client monitors mic energy. Above threshold for ~150 ms means it stops playback immediately and sends `interrupt { turn_id, played_ms }`.
- Server cancels the LLM and TTS tasks for that turn, then maps `played_ms` to sentence boundaries using the audio duration of each sent sentence. History keeps only the sentences actually heard (the last one is trimmed if partially played), and the turn is stored with `interrupted = true`.
- Baseline if `played_ms` mapping is not finished in time: keep only the sentences whose audio had been fully sent.

6. API and WebSocket event schema

REST

Endpoint	Request	Response
`POST /sessions`	`{}`	`{ "session_id": "uuid" }`
`GET /sessions/:id`	none	`{ session, turns, metrics, errors }`
`GET /healthz`	none	`{ "ok": true }`

WebSocket `/ws/voice?session_id=<uuid>`

Binary frames from client to server are raw PCM (D2). Binary frames from server to client are raw PCM audio chunks (D3), always inside an `agent_audio_start` / `agent_audio_end` pair. All other messages are JSON text frames of the form `{ "type": "...", ... }` with a server `ts` (epoch ms) on server messages.

Client → server

type	Payload	When
(binary)	PCM s16le 16 kHz mono	Continuously while call is active
`interrupt`	`{ turn_id, played_ms }`	User spoke over the agent
`playback_done`	`{ turn_id }`	Agent audio finished playing
`end_call`	`{}`	User clicked End Call
`ping`	`{}`	Keepalive every 15 s

Server → client

type	Payload	Purpose
`state`	`{ state }`	listening / thinking / speaking / ended
`transcript_partial`	`{ text }`	Live captions
`transcript_final`	`{ text, turn_id }`	Committed user turn
`agent_text`	`{ text, turn_id, sentence_index }`	Agent caption per sentence
`agent_audio_start`	`{ turn_id, sentence_index, sample_rate, encoding: "pcm_s16le", channels: 1 }`	Announces audio format for the chunks that follow
(binary)	PCM audio chunk	Agent speech
`agent_audio_end`	`{ turn_id, sentence_index }`	End of that sentence's audio
`turn_done`	`{ turn_id, total_audio_ms }`	Agent reply fully sent

type	Payload	Purpose
`tool_event`	`{ turn_id, name, status: "started" or "ok" or "error" }`	Optional UI hint
`error`	`{ code, message, recoverable }`	See section 9
`call_ended`	`{ reason }`	Session closed
`pong`	`{}`	Keepalive reply

`reason` values: `user_end`, `silence_timeout`, `max_duration`, `ws_drop`, `server_error`.

Example flow

```
{"type":"state","state":"listening"}
{"type":"transcript_partial","text":"what time"}
{"type":"transcript_final","text":"What time is it in Tokyo?","turn_id":3}
{"type":"state","state":"thinking"}
{"type":"tool_event","turn_id":3,"name":"get_current_time","status":"started"
}
{"type":"agent_text","text":"It is 4:10 PM in
Tokyo.","turn_id":3,"sentence_index":0}
{"type":"state","state":"speaking"}
{"type":"agent_audio_start","turn_id":3,"sentence_index":0,"sample_rate":2400
0,"encoding":"pcm_s16le","channels":1}
```

IDs: `turn_id` is an integer incremented per turn (user and agent turn share the id of the exchange). Every JSON event from the server carries it where relevant so the client can ignore late events from a cancelled turn.

7. Metrics and latency measurement

Timestamps are captured server-side with a monotonic clock per turn:

- `t_speech_start`, `t_vad_end` (when VAD declared end of utterance), `t_stt_final`, `t_llm_first_token`, `t_tts_first_audio` (first byte of audio sent to client).
- `vad_end_ms = silence_threshold` (the silence wait), `stt_final_ms = t_stt_final - t_vad_end`, `llm_first_token_ms = t_llm_first_token - t_stt_final`, `tts_first_audio_ms = t_tts_first_audio - t_first_sentence_ready`.

- `total_ms = t_tts_first_audio - (t_vad_end - silence_threshold)`: estimated time from the user's last spoken word to first agent audio, as the assignment budget defines it. Also record `total_ms_from_vad_end` so numbers are comparable either way.
- Server-side timing excludes network and browser audio-output delay. The client reports `first_audio_played_at` in `playback_done` so the report can state both.

A benchmark script replays a recorded audio file N times through the full pipeline and prints median and p90 per stage. That script also serves as the integration test and the source for the latency report.

8. Persistence (SQLite)

```
CREATE TABLE sessions (
  id TEXT PRIMARY KEY, started_at TEXT, ended_at TEXT, end_reason TEXT,
  config_json TEXT  -- providers, prompt version, thresholds
);
CREATE TABLE turns (
  session_id TEXT, turn_id INTEGER, role TEXT,        -- user | agent
  text TEXT, interrupted INTEGER DEFAULT 0,
  tool_calls_json TEXT,                              -- calls + results
  PRIMARY KEY (session_id, turn_id, role)
);
CREATE TABLE turn_metrics (
  session_id TEXT, turn_id INTEGER,
  vad_end_ms REAL, stt_final_ms REAL, llm_first_token_ms REAL,
  tts_first_audio_ms REAL, total_ms REAL,
  PRIMARY KEY (session_id, turn_id)
);
CREATE TABLE errors (
  session_id TEXT, turn_id INTEGER, stage TEXT, code TEXT,
  message TEXT, ts TEXT
);
```

Writes happen at turn end and at session end. A WS drop still triggers the session-end write so the transcript is saved.

9. Error handling matrix

Failure	Detection	Behavior	Code
Mic permission denied	Client `getUserMedia` rejects	UI message, no session created	client-only
STT error or timeout	Adapter exception or 3 s no final	Speak fallback line, log, back to LISTENING	`STT_ERROR`
LLM error or timeout	Exception or 5 s no first token	Speak fallback line, log, back to LISTENING	`LLM_ERROR`
TTS error or timeout	Exception or 5 s no first audio	Send `agent_text` only, show caption, log, back to LISTENING	`TTS_ERROR`
Tool failure	Tool raises or times out	Error JSON returned to LLM, which answers gracefully	`TOOL_ERROR`
Empty or noise-only utterance	VAD min speech length (default 250 ms) or empty STT final	Ignore, no LLM call	none
WebSocket drop	Socket closed or no ping for 30 s	Mark ended with `ws_drop`, persist; client shows Disconnected	`WS_DROP`
Silence timeout (120 s) / max duration	Session timers	Short goodbye, then `call_ended`	none

Fallback line: "Sorry, I didn't catch that, could you repeat?" (configurable). If TTS itself is down, the fallback is shown as text only.

10. Configuration

Environment variables with defaults: `SYSTEM_PROMPT`, `PROMPT_VERSION`, `VAD_SILENCE_MS=600`, `VAD_MIN_SPEECH_MS=250`, `MAX_CALL_SECONDS=600`, `SILENCE_TIMEOUT_SECONDS=120`, `STT_PROVIDER`, `LLM_PROVIDER`, `TTS_PROVIDER`, per-

provider API keys, `STT_TIMEOUT_S`, `LLM_TIMEOUT_S`, `TTS_TIMEOUT_S`, `DB_PATH`. A `.env.example` ships with the repo.

11. Tools

- `get_current_time(timezone: str)` returns `{ iso, human }`. Proves the tool-calling path.
- `get_order_status(order_id: str)` returns mock data from a dict, and a not-found error for unknown ids (exercises the failure path).

Tool results are appended to history as tool messages and stored in `turns.tool_calls_json`.

12. Repo structure

```
voice-agent/
  backend/
    app/ main.py, api.py, ws.py, session.py, pipeline.py, sentence_buffer.py,
         metrics.py, config.py, db.py
    app/adapters/ stt_*.py, llm_*.py, tts_*.py, registry.py
    app/tools/ time_tool.py, order_tool.py
    tests/ unit + integration
  frontend/ index.html, app.js, mic-worklet.js
  scripts/ benchmark.py
  docs/ design.md, latency_report.md
  README.md, .env.example
```

13. Testing plan

- Unit: state machine transitions (table in section 4), sentence buffer, each adapter with mocked network, tool executor, metrics math.
- Integration: recorded WAV file through VAD, STT, LLM, TTS (real or stubbed providers) asserting events and that a session is persisted.
- Manual acceptance run: 5+ turn conversation, barge-in test, tool test, failure injection for each row of section 9.

14. Risks and open questions

- Echo causing false barge-in: mitigate with browser echo cancellation and headphones for the demo.
- Free-tier quotas or rate limits mid-testing: keep a local fallback adapter (Piper, Vosk) ready.
- Streaming STT final-transcript delay: measure early; tune VAD silence downward if the budget is missed.
- Open: confirm providers after checking current free tiers. Open: confirm Python vs another backend language.

15. Milestone plan

M1 design + WebSocket echo (Oct 23–24), M2 happy path non-streaming (Oct 25), VAD + session manager (Oct 26), M3 streaming + captions + sentence-level TTS (Oct 27), barge-in + tool + history (Oct 28), error handling + persistence (Oct 29), tests + benchmark + one extra feature (Oct 30), README + demo + report (Oct 31), buffer and submission (Nov 1).